

CSA1717-ARTIFICIAL INTELLIGENCE

PRACTICAL LAB EXPERIMENTS

RAGAVI S – (192565027)

EXPERIMENT 5: CRIPT ARITHMETIC PROBLEM

Aim

To understand and implement a solution for a Cryptarithmic puzzle using a constraint satisfaction approach in Python.

Objective

1. **To Solve Specific Puzzles:** Develop an algorithm that can find the digit assignments for a given Cryptarithmic addition puzzle (e.g., TWO + TWO = FOUR, or SEND + MORE = MONEY).
2. **Constraint Satisfaction:** To explore how problems with logical constraints can be solved through systematic searching and constraint checking.
3. **Backtracking:** Implement a search method (backtracking) that explores permutations of digits (0-9) to find the correct assignment while respecting rules (like no leading zeros).

Algorithm

Step 1: Define and read the input strings representing the cryptarithmic puzzle (Addend 1, Addend 2, and Result Sum).

Step 2: Extract all unique letters from the input strings and identify the leading letter of each word to enforce the non-zero leading digit rule.

Step 3: Check if the count of unique letters exceeds 10; if true, display "No Solution Possible" and stop, as only digits 0 through 9 are available.

Step 4: Initialize a recursive backtracking process to map each unique letter to a distinct digit from 0 to 9.

Step 5: Evaluate constraints for the current digit assignment: if a leading letter is assigned the digit 0 or if a digit is repeated, reject the assignment and backtrack to pick the next available digit combination.

Step 6: Convert each string into its corresponding numeric value by replacing every letter with its assigned digit.

Step 7: Verify whether the arithmetic equation holds true (Numeric Addend 1 + Numeric Addend 2 = Numeric Result Sum).

Step 8: If the equation is satisfied, print the valid letter-to-digit mapping along with the solved numerical equation; otherwise, backtrack to attempt remaining digit combinations until a solution is found or all possibilities are exhausted, then stop the program.

Source Code

```
import itertools
```

```
def solve_cryptarithmic(addend1, addend2, result_sum):
```

```
    """Solves a Cryptarithmic puzzle using permutation-based constraint search.
```

```
    Rules:
```

1. Each letter represents a unique digit from 0 to 9.
2. The leading letter of a word cannot be zero.
3. The sum of the numeric values of addend1 and addend2 must equal result_sum.

```
    """
```

```
    # 1. Convert all words to uppercase
```

```
    addend1 = addend1.upper()
```

```
    addend2 = addend2.upper()
```

```
    result_sum = result_sum.upper()
```

```
    # 2. Identify unique letters and leading letters
```

```
    unique_letters = set(addend1 + addend2 + result_sum)
```

```
    leading_letters = set([word[0] for word in [addend1, addend2, result_sum]])
```

```
    # 3. Cryptarithmic constraint: Maximum 10 unique digits (0-9)
```

```
    if len(unique_letters) > 10:
```

```
print("[-] Invalid puzzle: More than 10 unique letters.")  
  
return
```

```
letters_list = tuple(unique_letters)  
  
digits = range(10)
```

```
print(f"[*] Analyzing puzzle: {addend1} + {addend2} = {result_sum}")  
  
print(f"[*] Unique letters ({len(letters_list)}): {''.join(letters_list)}")  
  
print(f"[*] Leading letters (non-zero): {''.join(leading_letters)}")  
  
print(f"[*] Searching for solutions...\n")
```

```
solution_count = 0
```

```
# 4. Generate permutations of digits matching the number of unique letters
```

```
for permutation in itertools.permutations(digits, len(letters_list)):  
  
    assignment = dict(zip(letters_list, permutation))
```

```
# 5. Reject assignments where any leading letter is 0
```

```
if any(assignment[letter] == 0 for letter in leading_letters):  
  
    continue
```

```
# Helper function to convert a word into its numerical value
```

```
def get_value(word):  
  
    num = 0  
  
    for letter in word:  
  
        num = num * 10 + assignment[letter]  
  
    return num
```

```
num1 = get_value(addend1)
num2 = get_value(addend2)
num_sum = get_value(result_sum)
```

```
# 6. Check arithmetic constraint
```

```
if num1 + num2 == num_sum:
```

```
    solution_count += 1
```

```
    print(f"[+] VALID SOLUTION FOUND (Solution #{solution_count}):")
```

```
    print(f"    Mapping: {assignment}")
```

```
    print(f"    Equation: {num1} + {num2} = {num_sum}\n")
```

```
if solution_count == 0:
```

```
    print("[-] Exhausted search space. No solution exists for this puzzle.")
```

```
if __name__ == "__main__":
```

```
    # Example 1: TWO + TWO = FOUR
```

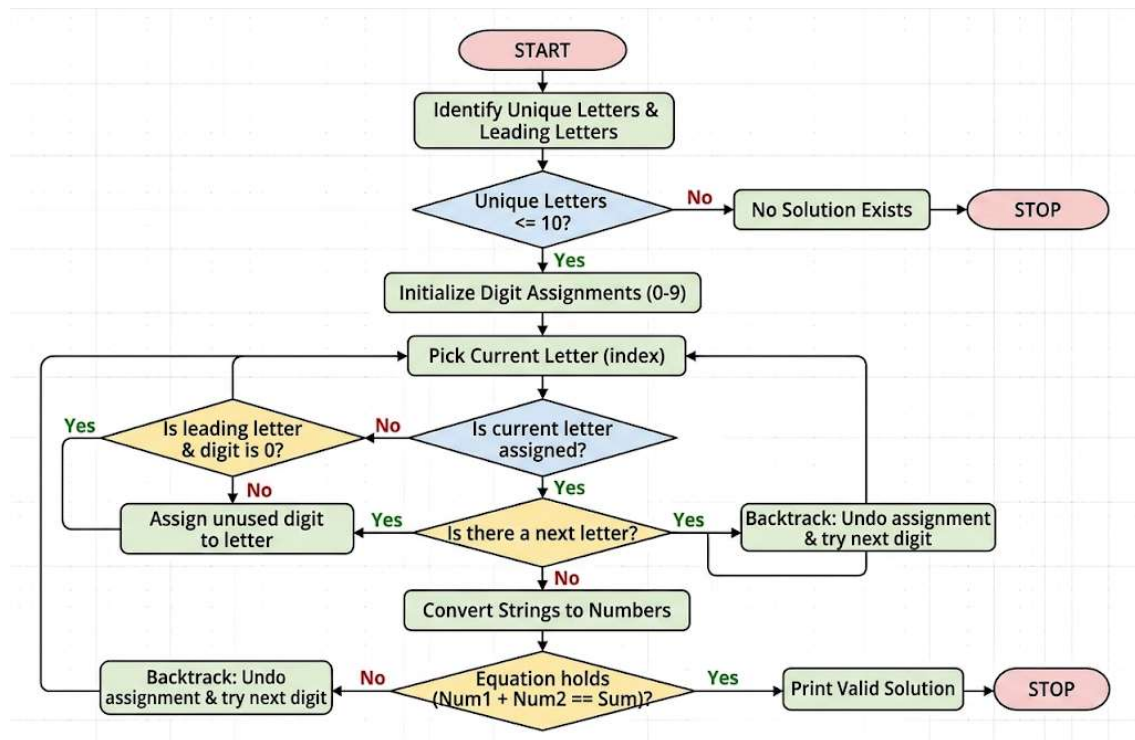
```
    solve_cryptarithmic("TWO", "TWO", "FOUR")
```

```
    print("=" * 50)
```

```
    # Example 2: SEND + MORE = MONEY
```

```
    solve_cryptarithmic("SEND", "MORE", "MONEY")
```

Flowchart



Output

```
=====
[*] Analyzing puzzle: SEND + MORE = MONEY
[*] Unique letters (8): SYMRNOED
[*] Leading letters (non-zero): MS
[*] Searching for solutions...

[+] VALID SOLUTION FOUND (Solution #1):
Mapping: {'S': 9, 'Y': 2, 'M': 1, 'R': 8, 'N': 6, 'O': 0, 'E': 5, 'D': 7}
Equation: 9567 + 1085 = 10652
>>>
```

Result

The Cryptarithmic problem was successfully simulated, and the permutation-based search agent efficiently identified valid digit-letter mappings satisfying the arithmetic constraints for both $TWO+TWO=FOUR$ and $SEND+MORE=MONEY$. Thus, the constraint satisfaction approach was implemented and verified correctly in Python.